THE PRINCIPLES OF DYNAMIC PROGRAMMING

Dynamic programming is one of the most influential paradigms in algorithm design, providing a structured way to solve optimization and decision problems that exhibit both *optimal substructure* and *overlapping subproblems*. Unlike divide-and-conquer, where subproblems are solved independently, dynamic programming identifies repeated computations and avoids redundancy by solving each distinct subproblem once and storing the results. Its power lies in converting exponential complexity into polynomial time by exploiting this inherent structure within a problem.

At the heart of dynamic programming is the recognition that many optimization problems contain smaller versions of themselves, and that the solutions to these smaller versions combine naturally to form solutions to larger ones. However, these subproblems frequently overlap: the same subinstances arise repeatedly during computation. Handling these repetitions naively causes inefficiency; dynamic programming confronts this directly by storing previously computed results in tables or arrays. This eliminates redundant calculations and enables efficient bottom-up or memoized top-down formulations.

The dynamic programming approach can be distilled into three fundamental steps. First, the structure of the optimal solution must be identified — a precise decomposition that reveals how the larger problem depends on smaller, similar subproblems. Second, a recurrence relation must be formulated. This recurrence quantifies the relationship between the problem and its subproblems, capturing the cost or value of transitions between states. Finally, the recurrence must be solved efficiently, usually by filling a table iteratively or through memoized recursion. This methodology transforms conceptual structure into concrete computation.

A recurring hallmark of dynamic programming is the use of *state definitions* to capture partial information. Each state represents a subproblem, and typically uses parameters that describe the portion of the input under consideration. For example, problems involving sequences often define states using indices; problems involving sets may define states through subsets or masks; problems involving alignment or transformation often use two-dimensional state grids. These states encode the minimal information needed to solve a subproblem independently.

Dynamic programming also emphasizes the distinction between *top-down* and *bottom-up* approaches. The top-down technique employs recursion with memoization: states are computed only when needed and their results stored to prevent recomputation. The bottom-up method constructs solutions iteratively, evaluating smaller states before larger ones and building up an entire table of values. Although both methods compute the same results, the bottom-up approach often provides superior clarity when dependencies form predictable patterns, while the top-down approach offers convenience and easier expression for complex recurrences.

Dynamic programming differs from greedy strategies in a critical way. While greedy algorithms make decisions based solely on local information, dynamic programming explores all possibilities at the subproblem level and guarantees global optimality through exhaustive yet efficient computation. This difference becomes especially clear in problems such as matrix chain multiplication and edit distance, where no local heuristic can guarantee

optimality — but dynamic programming, by examining all feasible substructures, ensures the optimal solution.

An essential component of successful dynamic programming is the careful formulation of recurrence relations. These relations must accurately embody the choices available in a subproblem and capture the cost or benefit associated with each choice. In many cases, the recurrence relation exposes combinational branching: a choice at one level leads to multiple possible subproblems. The dynamic programming table ensures that even with such branching, each individual subproblem is computed only once.

Beyond simple optimization, dynamic programming supports reconstructing actual solutions, not merely computing optimal values. This is done by augmenting tables with additional information — such as parent pointers, split points, or traceback matrices — that record the choices made during the computation. These mechanisms not only confirm optimality but also reveal the sequence of decisions that yield the optimal solution.

The versatility of dynamic programming appears in its wide range of applications. Problems involving shortest paths, string comparison, matrix parenthesization, sequence alignment, scheduling, partitioning, and combinatorial counting all lend themselves to dynamic programming formulations. What unites them is the combination of substructure that composes well and overlapping components that can be reused.

In this module, the exploration begins with matrix chain multiplication, a classic demonstration of how dynamic programming transforms exponential search spaces into polynomial-time solutions. The problem illustrates how optimal substructure is identified, how recurrence relations are constructed, and how a two-dimensional table is filled to compute the minimal cost of matrix multiplication. From there, the module proceeds into longest increasing subsequences, edit distance, and related dynamic techniques, reinforcing the systematic approach that lies at the core of dynamic programming.

Dynamic programming, therefore, is not a single algorithm but a disciplined methodology. It requires recognizing structure, formulating state and recurrence relations, and executing computation efficiently. Mastering these skills equips one to attack some of the most challenging problems in algorithmic theory and practice.

MATRIX CHAIN MULTIPLICATION

Matrix chain multiplication is one of the most influential illustrations of dynamic programming, demonstrating how exponential brute-force processes can be reorganized into efficient polynomial-time solutions by identifying the right structure. The problem arises when multiplying a sequence of matrices A1,A2,…,AnA_1, A_2, \dots, A_nA1,A2,…,An. Because matrix multiplication is associative, the product

A1A2A3⋯AnA_1 A_2 A_3 \cdots A_nA1A2A3⋯An

can be parenthesized in many different ways. Each valid parenthesization yields the same numerical result but may require a dramatically different number of scalar multiplications. Matrix chain multiplication asks: **what is the most cost-efficient way to parenthesize the product?**

To appreciate the underlying difficulty, consider three matrices A1,A2,A3A_1, A_2, A_3A1 ,A2,A3 whose dimensions allow multiplication. There are two ways to parenthesize the product:

(A1A2)A3orA1(A2A3).(A_1 A_2) A_3 \quad \text{or} \quad A_1 (A_2 A_3).(A1A2)A3 orA1(A2A3).

Although both formats yield the same final matrix, the number of scalar operations required can differ significantly. The cost of multiplying a p×qp \times qp×q matrix by a q×rq \times rq×r matrix is pqrpqrpqr. When extended to a long chain of matrices, the order of operations can influence the total number of operations by orders of magnitude. This makes determining the optimal parenthesization crucial for efficient computation.

The complexity arises from the number of ways to parenthesize the chain. The number of possible parenthesizations grows according to the Catalan numbers — for instance, 4 matrices yield 5 possible parenthesizations, 5 matrices yield 14, and 10 matrices already produce 17,000 alternatives. Enumerating these possibilities explicitly is infeasible for large chains. However, the structure of the problem reveals a powerful property: **optimal substructure**. Any optimal parenthesization of the entire chain must use optimal parenthesizations for its subchains. This observation is the foundational insight that makes dynamic programming applicable.

To formulate the recurrence, suppose matrix AiA_iAi has dimensions pi−1×pip_{i-1} \times p_ipi−1×pi. Then the cost of multiplying a chain from AiA_iAi to AjA_jAj depends on where the chain is split. Consider a split at position kkk, where i≤k<ji \le k < ji≤k<j. The left subchain is AiA_iAi through AkA_kAk, and the right subchain is Ak+1A_{k+1}Ak+1 through AjA_jAj. The cost of multiplying the two resulting matrices (after recursively multiplying the left and right chains) is:

pi−1·pk·pj.p_{i-1} \cdot p_k \cdot p_j.pi−1·pk·pj.

Thus, the total cost for this split is:

m[i,k]+m[k+1,j]+pi−1pkpj,m[i, k] + m[k+1, j] + p_{i-1} p_k p_j,m[i,k]+m[k+1,j]+pi−1pkpj,

where m[i,j]m[i, j]m[i,j] denotes the minimal number of scalar multiplications needed to compute the product AiAi+1⋯AjA_i A_{i+1} \cdots A_jAiAi+1⋯Aj.

The dynamic programming recurrence follows naturally:

m[i,j]={0,if i=jmin⁡i≤k<j(m[i,k]+m[k+1,j]+pi−1pkpj),if i<j.m[i, j] = \begin{cases} 0, & \text{if } i = j \\ \min_{i \le k < j} \left(m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \right), & \text{if } i < j. \end{cases}m[i,j]={0,mini≤k<j(m[i,k]+m[k+1,j]+pi−1pkpj),if i=jif i<j.

This recurrence captures the combinatorial branching of all possible ways to split the chain and computes the optimal one by examining all split positions. The value m[i,j]m[i, j]m[i,j] depends on smaller subproblems m[i,k]m[i, k]m[i,k] and m[k+1,j]m[k+1, j]m[k+1,j], ensuring the substructure aligns perfectly with dynamic programming principles.

The need for efficient computation becomes evident when observing the recurrence's natural recursive form. If implemented directly through naive recursion, the number of calls explodes exponentially because the same subproblems are recomputed repeatedly. Dynamic programming resolves this by storing results in a two-dimensional table, computing entries in order of increasing chain length. Such a table has size n×nn \times nn×n, and each entry requires evaluating O(n)O(n)O(n) split positions, yielding an overall time complexity of O(n3)O(n^3)O(n3), which is efficient for moderately large nnn.

Beyond computing the minimum cost, matrix chain multiplication also reveals the necessity of reconstructing the actual optimal parenthesization — not merely its cost. To accomplish this, a secondary table is maintained that records the optimal split point kkk for each interval [i,j][i, j][i,j]. Once both tables are filled, the complete parenthesization can be recovered via traceback, using the stored split points to reconstruct the placement of parentheses recursively.

Matrix chain multiplication encapsulates the core philosophy of dynamic programming:

- identify a clear substructure,
- derive a recurrence that ties larger problems to smaller ones,
- build a table to compute each subproblem exactly once,
- and, if needed, use additional information for reconstructing the full optimal solution.

It remains one of the classical examples because it balances theoretical elegance with practical clarity. The recurrence is conceptually simple, the table construction orderly and systematic, and the advantages over brute force dramatic.

In the next part, the full dynamic programming table, its construction, the cost computations, and the reconstruction of parentheses will be explored in detail.

THE DYNAMIC PROGRAMMING TABLE FOR MATRIX CHAIN MULTIPLICATION

Having established the recurrence relation for matrix chain multiplication, the next step is to transform it into an efficient computational procedure. Dynamic programming accomplishes this by building a two-dimensional table that captures the optimal cost for every possible subchain. This allows each subproblem to be solved exactly once and ensures that all data needed for larger subproblems is already computed.

Consider a chain of matrices A1,A2,…,AnA_1, A_2, \dots, A_nA1,A2,…,An, where each matrix AiA_iAi has dimensions pi−1×pip_{i-1} \times p_ipi−1×pi. The DP table is an n×nn \times nn×n matrix mmm, where m[i][j]m[i][j]m[i][j] stores the minimal number of scalar multiplications needed to compute the product AiAi+1⋯AjA_iA_{i+1}\cdots A_jAiAi+1 ⋯Aj. When i=ji = ji=j, the value is zero because a single matrix requires no multiplication.

The table is filled in increasing order of chain length. First, all chains of length 1 are initialized to zero. Then chains of length 2 are evaluated, followed by chains of length 3, and so on, until the entire chain length nnn is processed. At each stage of filling the table, the algorithm considers all valid split points kkk between iii and jjj, evaluating the total cost of the split:

m[i][k]+m[k+1][j]+pi−1pkpj.m[i][k] + m[k+1][j] + p_{i-1} p_k p_j.m[i][k]+m[k+1][j]+pi−1pkpj.

The minimal value among all possible kkk-values becomes the entry m[i][j]m[i][j]m[i][j].

To recover the exact sequence of matrix multiplications — the optimal parenthesization — a companion table sss is maintained. The entry s[i][j]s[i][j]s[i][j] records the split point kkk that yields the minimal cost for the interval [i,j][i, j][i,j]. Once the table is complete, the parenthesization can be reconstructed recursively by referring to the stored split points. For example, if s[i][j]=ks[i][j] = ks[i][j]=k, the optimal parenthesization of the chain Ai⋯AjA_i \cdots A_jAi⋯Aj is:

(Ai⋯Ak)(Ak+1⋯Aj),(A_i \cdots A_k)(A_{k+1} \cdots A_j),(Ai⋯Ak)(Ak+1⋯Aj),

with each side parenthesized recursively using the split table.

The framework for constructing these tables is captured in the following dynamic programming formulation:

```
void matrixChainOrder(vector<int> &p, int n,
                      vector<vector<int>> &m,
                      vector<vector<int>> &s) {
    for (int i = 1; i <= n; i++)
        m[i][i] = 0;

    for (int len = 2; len <= n; len++) {
        for (int i = 1; i <= n - len + 1; i++) {
            int j = i + len - 1;
            m[i][j] = INT_MAX;

            for (int k = i; k < j; k++) {
                int cost = m[i][k] + m[k+1][j] + p[i-1] * p[k] * p[j];

                if (cost < m[i][j]) {
                    m[i][j] = cost;
                    s[i][j] = k;
                }
            }
        }
    }
}
```

This algorithm faithfully implements the recurrence, constructing the lower triangular DP table row by row, with complexity O(n3)O(n^3)O(n3). Once the table is filled, the optimal parenthesization is obtained through a recursive procedure such as:

```
void printOptimal(int i, int j, vector<vector<int>> &s) {
    if (i == j)
        cout << "A" << i;
    else {
        cout << "(";
        printOptimal(i, s[i][j], s);
        printOptimal(s[i][j] + 1, j, s);
        cout << ")";
    }
}
```

This prints the multiplication order exactly as encoded by the DP table.

An important conceptual tool in understanding MCM is the **recursion tree**. The naive recursive formulation of MCM branches at every possible split point kkk, generating a tree whose size grows exponentially. Each node represents a subproblem (i,j)(i, j)(i,j), and each branch represents trying a particular split. The dynamic programming algorithm effectively *collapses this tree* into a compact table by avoiding recomputation. Every node in the recursion tree corresponds to a cell m[i][j]m[i][j]m[i][j]; dynamic programming ensures each such cell is computed exactly once. Visualizing this transformation underscores why DP is essential: without memoization or table-driven computation, the branching structure quickly becomes unmanageable.

The final DP table not only gives the optimal cost but also encodes all the substructure that leads to the best parenthesization. Each cell records the best decision for its interval, and the split table reveals the exact structural decomposition of the optimal multiplication strategy. Together, they turn a problem with combinatorial explosion into one with predictable and manageable complexity.

Matrix chain multiplication, with its clean recurrence, structured table, and clear traceback path, is thus a model example for understanding how dynamic programming reorganizes computation. It illustrates how problems that initially appear intractable can be tackled systematically by reframing them in terms of overlapping subproblems and optimal substructures.

ESSENTIAL ELEMENTS OF DYNAMIC PROGRAMMING

Dynamic programming succeeds because it recognizes and exploits two deep structural features inherent in many optimization and decision problems: **optimal substructure** and **overlapping subproblems**. These properties are not superficial; they lie at the core of algorithmic efficiency. Understanding them clearly is essential for designing correct DP formulations.

Optimal substructure means that an optimal solution to a problem can be constructed from optimal solutions of its subproblems. This is not merely a convenient fact — it is the defining characteristic that enables a recurrence relation to exist. When a problem breaks cleanly into smaller components whose solutions combine directly into a global solution, dynamic programming becomes a natural tool. In matrix chain multiplication, for example, the optimal way to multiply matrices AiA_iAi through AjA_jAj depends on the optimal way to multiply AiA_iAi through AkA_kAk and the optimal way to multiply Ak+1A_{k+1}Ak+1 through AjA_jAj. The recurrence captures this decomposition, and the DP table computes each subproblem exactly once.

Overlapping subproblems complement optimal substructure by revealing redundancy in naive recursion. A brute-force recursive approach often revisits the same subproblems again and again as the recursion tree expands. Dynamic programming removes this inefficiency by storing results in a table so that each subproblem is solved a single time. This transformation is why exponential-time recursions can collapse into polynomial-time DP solutions. The same phenomenon appears across diverse problems: the edit distance between two strings, the longest increasing subsequence, shortest paths in weighted graphs, and many others all revisit identical subinstances many times unless memoization or tabulation intervenes.

A successful DP formulation requires more than recognizing these two properties. The designer must also construct a **state representation** that captures exactly the information needed to answer a subproblem and nothing more. States must be precise and minimal: too little information makes the recurrence impossible to express, while too much creates unnecessary complexity and enlarges the DP table. In MCM, the state (i,j)(i, j)(i,j), representing the subchain from AiA_iAi to AjA_jAj, is perfectly chosen: it describes the exact portion of the chain whose optimal cost is required, and no additional detail is needed.

Once the state is chosen, the next step is to define the **recurrence relation**. This relation describes how to compute each state in terms of smaller states. Recurrences often arise by exploring all feasible choices at a decision point and selecting the best outcome. In many problems, this corresponds to iterating over possible partition points, transitions, or choices available at each substructure boundary. The recurrence must be mathematically correct and complete, describing all possible ways to form the state.

With the recurrence defined, dynamic programming proceeds either top-down or bottom-up. In a top-down approach, recursion is used, and memoization ensures subproblems are computed only once. In a bottom-up approach, the DP table is filled iteratively, typically in order of increasing subproblem size. The bottom-up technique removes recursion overhead and makes the computation order clear, though it may fill some states that are never needed. Both approaches rely on the same recurrence; the choice depends on convenience, clarity, and structure of the states.

Another important aspect of dynamic programming is the ability to **recover the actual solution**, not just its value. For many problems, including matrix chain multiplication and edit distance, the DP table stores auxiliary information — split points, choices, directions — that encode how the optimal value was achieved. This allows traceback procedures to reconstruct the full sequence of decisions. These tracebacks demonstrate how DP algorithms not only compute optimal values but also provide explicit optimal strategies.

Dynamic programming often reveals hidden symmetry and structure within complex problems. Sparse DP tables indicate that some states are inherently unreachable or unnecessary; dense tables show full interconnectedness. Dependencies between states show where constraints tighten or loosen. Such structure not only guides efficient implementation but also deepens conceptual understanding.

Finally, dynamic programming emphasizes the distinction between *solving a problem* and *formulating its structure*. Formulating the recurrence is frequently the most challenging intellectual step. Once the recurrence is properly defined, writing the DP algorithm becomes mechanical. This is why DP is sometimes regarded as a way of thinking rather than simply a class of algorithms. The shift from recursive intuition to tabulated structure requires precise reasoning and careful identification of subproblem boundaries.

Across all its applications — matrix chain multiplication, longest increasing subsequences, edit distance, knapsack variants, scheduling, and more — dynamic programming embodies this overarching methodology. It transforms brute force into elegance by changing how the problem is conceptualized, not just how it is computed.

LONGEST INCREASING SUBSEQUENCE (LIS)

The longest increasing subsequence problem provides one of the most elegant demonstrations of how dynamic programming captures the structure of sequential decision-making. Given a sequence of numbers, the objective is to determine the longest subsequence in which each element is strictly larger than the preceding one. Unlike contiguous subarrays, a subsequence does not require the elements to be adjacent — only ordered. This flexibility creates a wide search space, and naive approaches that examine all subsequences would require exponential time. Dynamic programming transforms this into a polynomial-time solution by uncovering the problem's intrinsic substructure.

Consider a sequence a1,a2,…,ana_1, a_2, \dots, a_na1,a2,…,an. The classical dynamic programming formulation introduces an array LIS[1..n]LIS[1..n]LIS[1..n], where LIS[i]LIS[i]LIS[i] represents the length of the longest increasing subsequence that **ends exactly at position** iii. This idea is the key to unlocking the recurrence. Instead of attempting to build subsequences arbitrarily, the DP approach fixes the endpoint and asks how increasing subsequences can converge to that point. Every valid subsequence ending at aia_iai must extend a previously computed subsequence ending at some aja_jaj with j<ij < ij<i and aj<aia_j < a_iaj<ai. This observation captures optimal substructure cleanly.

To compute LIS[i]LIS[i]LIS[i], the algorithm examines all earlier positions jjj and checks whether extending a subsequence ending at jjj would remain increasing. If so, the new subsequence ending at iii can have length

LIS[j]+1.LIS[j] + 1.LIS[j]+1.

The longest among all such possibilities determines LIS[i]LIS[i]LIS[i]. When no such jjj exists, the element aia_iai itself forms a subsequence of length 1. This reasoning leads directly to the recurrence relation:

LIS[i]=1+max⁤j<iaj<aiLIS[j],LIS[i] = 1 + \max_{\substack{j < i \\ a_j < a_i}} LIS[j],LIS[i]=1+j<iaj<aimax LIS[j],

with the understanding that the maximum over an empty set defaults to 0.

The DP procedure evaluates entries from left to right, ensuring that when computing LIS[i]LIS[i]LIS[i], all necessary values LIS[1],…,LIS[i−1]LIS[1], \dots, LIS[i-1]LIS[1],…,LIS[i−1] are already known. The final answer is the maximum value in the LISLISLIS array. A standard implementation follows naturally:

```
int LIS(vector<int> &a) {
    int n = a.size();
    vector<int> dp(n, 1);

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (a[j] < a[i])
                dp[i] = max(dp[i], dp[j] + 1);
        }
    }

    return *max_element(dp.begin(), dp.end());
```

```
}
```

This dynamic programming solution runs in O(n2)O(n^2)O(n2) time and is both intuitive and reliable. Each element is examined against all previous elements, and feasibility is determined by comparing values. If the sequence is long, this approach may not be optimal, but for moderate sizes it is perfectly effective and illustrates the fundamental idea of building solutions incrementally.

Beyond computing the length of the LIS, one often wants to reconstruct the actual subsequence. To achieve this, an auxiliary array `parent` is maintained, storing the index from which each `dp[i]` was derived. After filling the DP table, traceback begins at the index of the maximum value in the `dp` array and follows parent pointers backward until the beginning of the subsequence is reached. Reversing these collected elements yields the final increasing subsequence. This procedure mirrors the traceback used in matrix chain multiplication, reinforcing the idea that DP not only solves optimization problems but also provides a mechanism to reconstruct optimal decisions.

The longest increasing subsequence problem highlights a valuable principle: dynamic programming excels when the future depends on choices in the past but only through a compact summary of those choices. Here, the length of the longest subsequence ending at a particular index is all that matters; the detailed identity of the earlier elements only matters insofar as it influences the recurrence. This compactness allows the dynamic programming table to encode all necessary information for optimality.

More advanced methods, such as the patience sorting technique, solve LIS in O(nlog⁡n)O(n \log n)O(nlogn), but the DP formulation remains essential for clarity and understanding. It introduces the idea of building forward, comparing elements systematically, and deriving maximal choices efficiently. In the broader landscape of dynamic programming, LIS stands alongside problems like edit distance and matrix chain multiplication as a fundamental illustration of optimal substructure and incremental solution construction.

STRING EDITING AND THE EDIT DISTANCE PROBLEM

String editing, or edit distance computation, is one of the most significant applications of dynamic programming in text processing, pattern matching, computational linguistics, DNA sequence analysis, and error correction. The objective is to determine how similar two strings are by measuring the minimal number of operations required to transform one string into the other. The allowed operations typically include **insertion**, **deletion**, and **substitution**, each with an associated cost. The edit distance between two strings is defined as the minimum total cost of operations that convert one string into the other.

Let the two strings be

X=x1x2…xm,Y=y1y2…yn.X = x_1 x_2 \dots x_m, \qquad Y = y_1 y_2 \dots y_n.X=x1x2…xm,Y=y1y2…yn.

The problem requires computing the minimal cost of editing XXX so that it becomes exactly equal to YYY. The structure of this problem is inherently sequential; each character of the first string interacts with each character of the second through choices that reflect whether characters match or require modification. Dynamic programming is ideally suited for this

because the transformation of prefixes of XXX into prefixes of YYY breaks naturally into smaller subproblems that overlap extensively.

To capture the substructure formally, define a two-dimensional DP table D[0..m][0..n]D[0..m][0..n]D[0..m][0..n], where

D[i][j]=minimum cost to convert x1x2…xi into y1y2…yj.D[i][j] = \text{minimum cost to convert } x_1 x_2 \dots x_i \text{ into } y_1 y_2 \dots y_j.D[i][j]=minimum cost to convert x1x2…xi into y1y2…yj.

This state definition allows the problem to be decomposed into logical transitions. If the final operation transforms the prefix X[1..i]X[1..i]X[1..i] into the prefix Y[1..j]Y[1..j]Y[1..j], then one of three possibilities must hold:

1. **Delete xix_ixi**:
   Transform X[1..i−1]X[1..i-1]X[1..i−1] into Y[1..j]Y[1..j]Y[1..j], then delete the last character xix_ixi.
   The cost becomes:

   D[i−1][j]+del_cost.D[i-1][j] + \text{del_cost}.D[i−1][j]+del_cost.

2. **Insert yjy_jyj**:
   Transform X[1..i]X[1..i]X[1..i] into Y[1..j−1]Y[1..j-1]Y[1..j−1], then insert yjy_jyj at the end.
   The cost becomes:

   D[i][j−1]+ins_cost.D[i][j-1] + \text{ins_cost}.D[i][j−1]+ins_cost.

3. **Substitute xix_ixi with yjy_jyj** (or match if they are equal):
   Transform X[1..i−1]X[1..i-1]X[1..i−1] into Y[1..j−1]Y[1..j-1]Y[1..j−1], then either substitute (if xi≠yjx_i \neq y_jxi□=yj) or simply match (if they are equal).
   The cost is:

   D[i−1][j−1]+{0if xi=yjsub_costif xi≠yj.D[i-1][j-1] + \begin{cases} 0 & \text{if } x_i = y_j \\ \text{sub_cost} & \text{if } x_i \neq y_j.
   \end{cases}D[i−1][j−1]+{0sub_costif xi=yjif xi□=yj.

These rules yield the central recurrence relation:

D[i][j]=min⁡(D[i−1][j]+del,D[i][j−1]+ins,D[i−1][j−1]+sub or match).D[i][j] = \min \left(D[i-1][j] + \text{del}, D[i][j-1] + \text{ins}, D[i-1][j-1] + \text{sub or match} \right).D[i][j]=min(D[i−1][j]+del,D[i][j−1]+ins,D[i−1][j−1]+sub or match).

The boundary conditions reflect the costs of converting empty prefixes. Converting an empty string to a string of length jjj requires jjj insertions, and converting a string of length iii to an empty string requires iii deletions:

D[0][j]=j⋅ins,D[i][0]=i⋅del.D[0][j] = j \cdot \text{ins}, \qquad D[i][0] = i \cdot \text{del}.D[0][j]=j⋅ins,D[i][0]=i⋅del.

Using these relationships, the DP table is filled row by row or column by column until the value D[m][n]D[m][n]D[m][n] — the cost to convert the entire string XXX into YYY — is computed.

A typical implementation employs uniform costs for simplicity, often taking each insertion, deletion, and substitution to cost 1. The C++ code below illustrates the DP computation:

```
int editDistance(string X, string Y) {
    int m = X.size(), n = Y.size();
    vector<vector<int>> D(m + 1, vector<int>(n + 1));

    for (int i = 0; i <= m; i++)
        D[i][0] = i;

    for (int j = 0; j <= n; j++)
        D[0][j] = j;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            int cost = (X[i - 1] == Y[j - 1]) ? 0 : 1;
            D[i][j] = min({
                D[i - 1][j] + 1,
                D[i][j - 1] + 1,
                D[i - 1][j - 1] + cost
            });
        }
    }

    return D[m][n];
}
```

This algorithm runs in O(mn)O(mn)O(mn) time and space, making it suitable for strings of moderate length. The DP table not only yields the minimum cost but also lays the groundwork for reconstructing the sequence of edits required to convert one string into the other.

To reconstruct the edit sequence, one traces back from D[m][n]D[m][n]D[m][n] to D[0][0]D[0][0]D[0][0] by observing which transition yielded the optimal value. If the current cell matches D[i−1][j]+delD[i-1][j] + \text{del}D[i−1][j]+del, then a deletion occurred. If it matches D[i][j−1]+insD[i][j-1] + \text{ins}D[i][j−1]+ins, then an insertion occurred. If it matches D[i−1][j−1]+costD[i-1][j-1] + \text{cost}D[i−1][j−1]+cost, then either a substitution or a match took place. Backtracking through these choices yields the actual series of editing operations — a pattern similar to optimal parenthesization and LIS reconstruction, reinforcing that DP is as much about recovering solutions as computing values.

The edit distance problem encapsulates all core themes of dynamic programming. The DP table embodies overlapping subproblems; each cell depends on three neighbors. The recurrence expresses optimal substructure clearly, balancing multiple feasible choices. The algorithm is efficient because it transforms exponential possibilities into a structured grid that is systematically filled. This problem’s formulation and solution serve as templates for more advanced sequence-alignment problems, including weighted edit distances, Levenshtein distance variations, and biological sequence alignment with scoring matrices.

DP CODE STRUCTURES AND MODULE CONCLUSION

Dynamic programming combines mathematical precision with structured computation, and its implementations across different problems follow recurring patterns. The examples in this module — matrix chain multiplication, longest increasing subsequence, and edit distance — demonstrate how DP transforms conceptual recurrences into efficient, tabulated algorithms. Despite differences in problem statements and dimensions, the code across these applications shares a consistent structural blueprint.

A characteristic aspect of dynamic programming code is the construction of a table (one-dimensional or two-dimensional) that stores intermediate results. For problems based on sequences, such as LIS, a one-dimensional table suffices; for problems involving transformations between two sequences, such as edit distance, a two-dimensional matrix becomes essential. For more complex problems like matrix chain multiplication, a two-dimensional DP table encodes optimal costs for subchains, while a companion table records optimal splitting decisions. In all cases, the goal remains the same: ensure that each subproblem is solved once and made available for reuse.

Across all the dynamic programming implementations in this module, three coding phases consistently appear. First, the DP table is initialized using clear base conditions. In edit distance, the first row and column reflect the cost of converting empty prefixes. In matrix chain multiplication, the diagonal entries represent the trivial cost of multiplying one matrix. In LIS, each element begins with a default value of one, signifying that any element by itself forms a subsequence of length one.

The second phase fills the DP table using the recurrence relations derived earlier. This is the computational heart of every DP algorithm. In matrix chain multiplication, nested loops enumerate chain lengths, starting indices, and splitting points. In LIS, each index scans all earlier indices to update subsequence lengths. In edit distance, the table is constructed by comparing adjacent characters and considering insertion, deletion, and substitution operations.

Finally, many dynamic programming solutions include a reconstruction phase. Beyond computing optimal values, many real-world problems require the optimal solution path itself. For matrix chain multiplication, the split table enables reconstruction of the parenthesization. For edit distance, backtracking through the DP matrix reveals the sequence of operations that converts one string into another. Even LIS can reconstruct its subsequence by tracing parent pointers. This capability underscores the dual advantage of dynamic programming: efficient computation and explicit decision recovery.

Here is a representative example of the matrix chain multiplication DP code in full, combining both table construction and optimal parenthesization:

```
void matrixChainOrder(vector<int> &p, int n,
                      vector<vector<int>> &m,
                      vector<vector<int>> &s) {
    for (int i = 1; i <= n; i++)
        m[i][i] = 0;

    for (int len = 2; len <= n; len++) {
        for (int i = 1; i <= n - len + 1; i++) {
```

```
            int j = i + len - 1;
            m[i][j] = INT_MAX;

            for (int k = i; k < j; k++) {
                int cost = m[i][k] + m[k+1][j] + p[i-1] * p[k] * p[j];

                if (cost < m[i][j]) {
                    m[i][j] = cost;
                    s[i][j] = k;
                }
            }
        }
    }
}

void printOptimal(int i, int j, vector<vector<int>> &s) {
    if (i == j)
        cout << "A" << i;
    else {
        cout << "(";
        printOptimal(i, s[i][j], s);
        printOptimal(s[i][j] + 1, j, s);
        cout << ")";
    }
}
```

This illustrates the classic DP workflow: initialize, build, and reconstruct.

Dynamic programming differs from greedy strategies and divide-and-conquer not in its reliance on recursion but in its treatment of overlapping subproblems and systematic reuse of results. Whereas greedy methods commit early to locally optimal choices and divide-and-conquer recomputes subproblems independently, dynamic programming unifies all subinstances into a holistic structure, eliminating redundancy and retaining global optimality. The conceptual clarity of DP comes from understanding how these subinstances fit together and how the recurrence naturally emerges from the problem's structure.

As this module has shown, DP is not simply a technique but a mode of analysis. It requires the solver to think deeply about how smaller problems relate to the larger one and how solutions can be combined. Once this structure is understood, the process of writing the DP becomes methodical.

Matrix chain multiplication demonstrates how an exponential number of parenthesizations can be handled efficiently by tabulating costs. LIS shows how DP simplifies sequential optimization. Edit distance reveals DP's strength in modeling transformation processes through local operations. Each reinforces the core ideas of subproblems, recurrence, tabulation, and traceback.

Together, these topics provide a thorough foundation for dynamic programming. Mastery of these principles prepares one to tackle advanced problems in sequence analysis, graph theory, scheduling, optimization, and algorithmic design. Dynamic programming remains one of the most versatile and intellectually rich approaches in the field of computer science, and its influence extends far beyond the examples contained within this module.